AI Agent Skills and MCP: Building the Right AI Agent Pattern

Stephen Simon

Chapter 4: Combining Agent Skills and MCP

Combining Agent Skills and MCP: Workflow Design Patterns

Reusable patterns from the final launch review workflow

In this chapter you combined the `launch-risk-review` Skill with the Orbit Labs MCP server and built a workflow that reads live data, judges it consistently, and records risks safely. This article collects the design patterns behind that workflow, so you can apply them to your own.

What you built in this chapter

- **Video 1:** brought the Skill back and added a Gather the information section, so the agent fetched live data in a consistent way and judged it with the team's rubric.
- **Video 2:** added rules for writing to the shared risk register, then ran the complete workflow twice with no duplicates.
- **Video 3:** applied the decision framework to other workflows and learned the signs of a mismatched pattern.

The final workflow runs from a one-line request: *Do this week's launch review.*

Design patterns

The Skill orchestrates, MCP provides. The Skill holds the process: when it applies, what to fetch, how to judge it, and what to do next. The server stays generic. The Orbit server exposes a project tracker and a support desk, not "the launch review."

Tell the Skill where to look. Naming the tools and filters in the Skill means the agent gathers the same data every time, instead of improvising a new plan on each run.

Fall back gracefully. Prefer live data when the server is available, use documents when it isn't, and say clearly when the result is based on a snapshot.

Read first, write last. Gather and judge before acting. Check what already exists before creating anything, write only what meets clear criteria, and report every change. That makes the workflow safe to run more than once.

Keep two layers of control. The Skill governs what the agent tries to do. Tool permissions govern what it's allowed to do. Use both, especially for tools that write to shared systems.

Keep the line clear. Tool descriptions say what a tool does. Skills say how your team works. If you find yourself writing policy into a tool description, it belongs in a Skill.

One Skill, swappable servers. Because the know-how and the access are separate, you can point the Skill's Gather the information section at a different server, such as a Jira or GitHub server, without touching the rubric, the template, or the write rules.

Review a hybrid workflow

Use these questions when you design a workflow that combines Skills and MCP:

1. Does the Skill's description name the situations where it should apply?
2. Does the Skill say which tools to call and with which filters?
3. Does the Skill say what to do when live data and documents disagree, or when the server isn't available?
4. Are severity, priority, or other judgment rules explicit and testable?
5. For every tool that writes: what qualifies, how are duplicates avoided, and how are changes reported?
6. Which tool calls require a person's approval?
7. Is any team policy hiding in a tool description?
8. If you moved to a different system, how much of the Skill would need to change?

Where to go next

- **Connect a real server.** Replace the Orbit server with one you use, such as the GitHub MCP server, and have the Skill create issues instead of risk items.
- **Add a second Skill.** Write a release notes Skill that reuses the same server with different know-how.
- **Test changes to your Skills.** Save a few reports you trust as expected results, and compare new versions of a Skill against them before sharing.